Competency 4: Uses logic gates to design basic digital circuits and devices.

Competency Level 4.1: Analyzes basic digital logic gates in term of their unique functionalities.

Number of periods: 06

Learning Outcomes:

- Names basic logic gates and draws the appropriate symbols of them.
- Draws the truth tables for the basic logic gates
- Identify symbols that represent that negations of basic logic gates.
- Creates truth tables for given expressions
- Explains the need of universal gates.
- Explain the fabrication of any gate.

Contents:

- Digital logic gates and truth tables
  - Basic logic gates
    - NOT, AND & OR
  - Combinational gates
    - NAND, NOR, XOR and XNOR
  - Universal gates
    - NAND, NOR

Concepts and terms need to be highlighted:

Brief discussion of the use of gates in computer hardware

Guidance for lesson plans:

- Explanation of the concept of gates using day-to-day examples.
- Simulation of gates using electrical circuits.
- Explanation of the concept of combinational and universal gates
- Final output of a combination of logic gates (eg. In ICs).

Guidance for assessments and evaluations

- Drawing gates and creation of truth tables for given logic circuits

Quality inputs:

- Presentations, open source software to demonstrate the gates and its functionality

Reading Material

Logic gate is an electronic component that performs basic logical functions and it can be identified as the fundamental building block of a digital circuit.

- **What are basic logical functions?**
  - Negation (NOT)- True if input is false, False if input is true
  - Conjunction (AND) – True if, A and B are both true
  - Disjunction (OR)- True if, either A or B are true
- **Basic logic gates**
  - NOT
  - AND
  - OR
- **Basic characteristics of logic gates**
  - Each logic gate has a unique symbol and a unique logical function.
  - A gate can have multiple input(s) and an output.
  - An input can take 0 or 1 at a time.
  - Input are processed according to the function of the particular logic gate and an output is provided.
  -

What is a truth table?

Truth table represents all the input combinations and corresponding output of a logic circuit.

The following table shows all possible input combinations for two and three inputs only.

No. of Inputs	Value combinations	No. of combinations
2	0 0 0 1 1 0 1 1	$4 = 2^2$
3	0 0 0 0 0 1 0 1 0 0 1 1 1 0 0 1 0 1 1 1 0 1 1 1	$8=2^3$

Therefore,

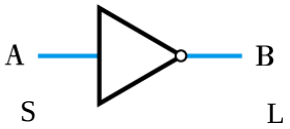
Number of input combination = $2^{\text{Number of inputs of logic gate/ circuit}}$

INTRODUCTION TO BASIC LOGIC GATES

- **NOT gate**
  - When switch A is in on state, bulb doesn't turn on. Bulb lights only if the Switch in off state.

S	L
0	1
1	0

Not gate has a single input and output. A NOT gate performs *logical negation* on its input. In other words, if the input is **true**, then the output will be **false**. Similarly, a **false** input results in a **true** output.



(a) Circuit symbol

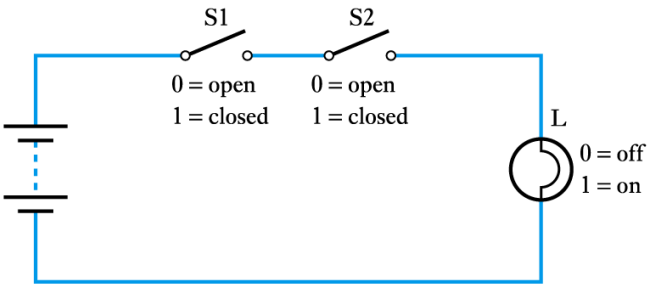
S	L
0	1
1	0

(b) Truth table

$L = \overline{S}$

(c) Boolean expression

- **AND gate**
  - In the following circuit, the bulb is lit if and only if both switches are in on state.
  - Light bulb is not lit if at least one switch is in off state.



(a) Circuit

L = S1. S2

S1	S2	L
0	0	0
0	1	0
1	0	0
1	1	1

(b) Truth table

The output of an AND gate is **true** only when all of the inputs are **true**. If one or more of an AND gate's inputs are **false**, then the output of the AND gate is **false**.

(a) Circuit symbol

S1	S2	L
0	0	0
0	1	0
1	0	0
1	1	1

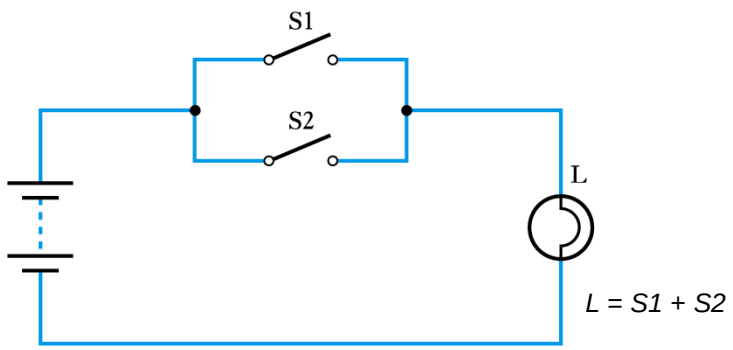
(b) Truth table

L = S1. S2

(c) Boolean expression

- **OR gate**

  In the following circuit, light bulb is lit if at least one switch is in ON state.



(a) Circuit

S1	S2	L
0	0	0
0	1	1
1	0	1
1	1	1

(b) Truth table

The output of an OR gate is true if at least one input is true. Otherwise output is false.

(a) Circuit symbol

S1	S2	L
0	0	0
0	1	1
1	0	1
1	1	1

(b) Truth table

L = S1+ S2

(c) Boolean expression

Other gates

- **Introduction to NAND gate**
  - In digital electronics, a **NAND gate** (NOT-AND) is a logic **gate** which produces false output which is false only if all its inputs are true; Otherwise it results a true output. thus, its output is the complement of AND **gate**.

(a) Circuit symbol

S1	S2	L
0	0	1
0	1	1
1	0	1
1	1	0

(b) Truth table

$L = \overline{S1.S2}$

(c) Boolean expression

- **NOR gate**
  - The **NOR gate** is a digital logic **gate** that results a true output if both inputs to the gate are false. Otherwise result is false. **NOR** is the result of the negation of the OR gate.

(a) Circuit symbol

S1 A	S2 B	L C
0	0	1
0	1	0
1	0	0
1	1	0

(b) Truth table

$L \quad \overline{S1.S2}$

C

(c) Boolean expression

- **XOR gate**
  - The **XOR gate** (pronounced as **Exclusive OR gate**) is a digital logic **gate** that results a true if and only one of the inputs to the gate is true. Otherwise output is false.

(a) Circuit symbol

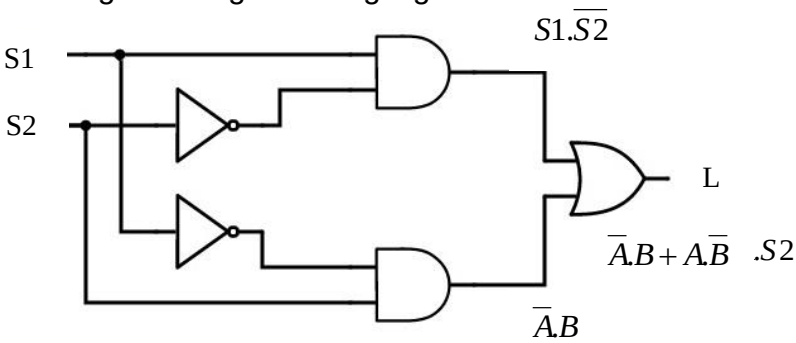
S1	S2	L
0	0	0
0	1	1
1	0	1
1	1	0

(b) Truth table

$L = S1 \oplus S2$

(c) Boolean expression

  - XOR gate using basic logic gates



S1	S2	$\overline{S1}$	$\overline{S2}$	$S1.\overline{S2}$	$\overline{S1}.S2$	$S1.\overline{S2}+\overline{S1}.S2$
0	0	1	1	0	0	0
0	1	1	0	0	1	1
1	0	0	1	1	0	1
1	1	0	0	0	0	0

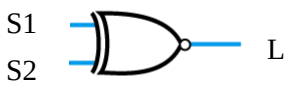
Truth table for three input XOR gate

INPUTS				**Final Output**
S1	S2	S3	S1 ⊕ S2	S1 ⊕ S2 ⊕ S3
0	0	0	0	0
0	0	1	0	1
0	1	0	1	1
0	1	1	1	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	0
1	1	1	0	1

- Compare S1 & S2 and if the inputs are same then output is zero (false). Otherwise output is one. Then compare the result of S1 ⊕ S2 with S3. if the values are same then final output is 0. else final output is 1.

Introduction to XNOR gate

- The **XNOR gate** is a digital logic gate whose function is the logical complement of the XOR gate.



(a) Circuit symbol

S1	S2	L
0	0	1
0	1	0
1	0	0
1	1	1

(b) Truth table

$L = \overline{S1 \oplus S2}$

(c) Boolean expression

Truth table for three input XNOR gate

INPUTS					**Final Output**
S1	S2	S3	S1 ⊕ S2	S1 ⊕ S2 ⊕ S3	$\overline{S1 \oplus S2 \oplus S3}$
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	1	1	0
0	1	1	1	0	1
1	0	0	1	1	0
1	0	1	1	0	1
1	1	0	0	0	1
1	1	1	0	1	0

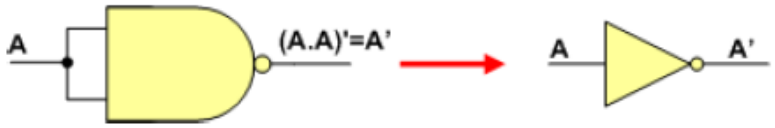
Universal Gates

A universal logic gate is a logic gate that can be used to construct all other logic gates. The NAND gate and NOR gates can be considered as universal logic gates.

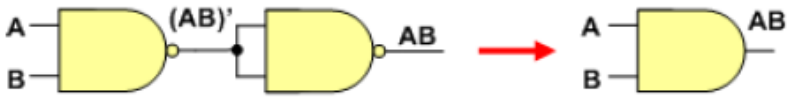
The advantage of universal gates: NAND and NOR gates are economical and easier to fabricate and are the basic gates used in all IC digital logic families.

NAND Gate

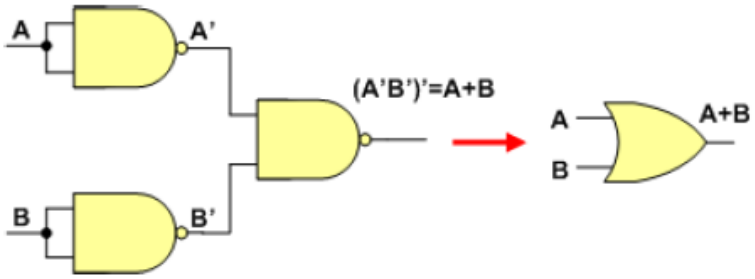
1. How to implement NOT gate using NAND gates?



2. How to implement AND gate using NAND gates?

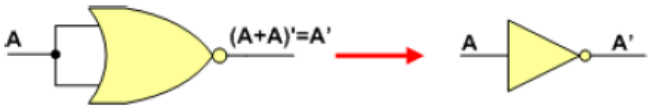


3. How to implement OR gate using NAND gates?

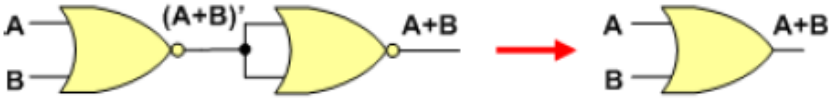


NOR Gate

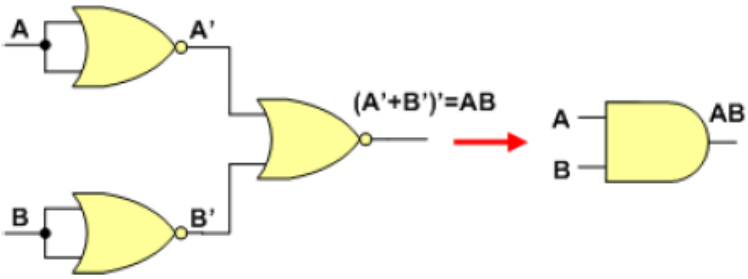
1. How to implement NOT gate using NOR gates?



2. How to implement OR gate using NOR gates?



3. How to implement AND gate using NOR gates?



Competency Level 4.2: Simplifies logic expressions using law of Boolean algebra and Karnaugh map.

Learning outcomes

- Describes the need for simplifying the Boolean expressions.
- Represents logical expressions in standard forms (SOP and POS) according to the given truth table
- Transforms SOP into POS and vice versa
- Simplifies logic expressions using Boolean theorems, axioms, De Morgan's Laws and Karnaugh map.

Number of periods: 08

Contents:

- Two state logic and Boolean algebra
- Postulates (Axioms)
- Law/ theorems
    - Commutative, associative
- Distributive
    - Identity, Redundancy
    - De Morgan's
- Standard logical expressions
    - Sum of Product and Product of Sum
    - Transforms SOP into POS and vice versa
- Simplify logic expressions using
    - Boolean theorems
    - Karnaugh maps

Concepts and terms need to be highlighted:

- Boolean algebra
- Postulates (Axioms)
- Law/ theorems
- Simplification of logic expressions
- Sum of Product and Product of Sum
- Transforms SOP into POS and vice versa

Guidance for lesson plans:

- Explanation of the concept of Boolean algebra using day-to-day examples.
- Explanation of Boolean algebra.
- Explanation and discussion of postulates and law/theorems
- Simplification of logic expressions using Boolean theorems and Karnaugh maps
- Explanation of SOP and POS
- Transformation of SOP into POS and vice versa

Guidance for assessments and evaluations

- Provide simple MCQ, structure and essay type questions on Boolean algebra
- provide logic expressions to simplify using postulates and law/theorems
- provide logic expressions to draw truth table
- Then ask the student to simply the expression using Karnaugh map
- provide a expression and ask the student to find the SOP and POS and then ask the students to transform SOP to POS and POS to SOP

Quality inputs:

- Presentations, open source software to demonstrate the gates and its functionality

Reading Material

Logical NOT, AND & OR Operators

In Boolean algebra, logical negation (NOT operation), logical conjunction (AND operation) and logical disjunction (OR operation) are performed using ¯ (bar), **.** (dot) and **+** (plus) operators respectively.

Introduction to Boolean laws

What is the requirement of Boolean algebra simplification?

Simplification of Boolean expressions reduces the number of operations and number of logic gates for the implementation.

Most laws exist in two forms. One is multiplicative form and the other is additive form. At the multiplication, variables in the boolean expression are multiplied by each other and at the addition, variables in the boolean expression are added to each other.

1. Idompotent Law

Law and its representation from a truth table

A.A=A

A	A	A.A
0	0	0
1	1	1

A+A=A

A	A	A+A
0	0	0
1	1	1

$\overline{A}.\overline{A} = \overline{A}$

A	$\overline{A}$	$\overline{A}$	$\overline{A}.A$
0	1	1	1
1	0	0	0

$\overline{A}+\overline{A} = \overline{A}$

A	$\overline{A}$	$\overline{A}$	$\overline{A}+\overline{A}$
0	1	1	1
1	0	0	0

Above highlighted columns are identical.

2. Identity Law

1.A=A

1	A	1.A
1	0	0
1	1	1

0+A=A

0	A	0+A
0	0	0
0	1	1

0.A = 0

0	A	0.A
0	0	0
0	1	0

1+A = 1

1	A	1+A
1	1	1
1	0	1

Above highlighted columns are identical.

3. Inverse/Complement Law

Multiplicative form

$A.\overline{A}=0$

A	$\overline{A}$	$A.\overline{A}$
0	1	0
1	0	0

Additive form

$A+\overline{A}=1$

A	$\overline{A}$	$A+\overline{A}$
0	1	1
1	0	1

4. De Morgan's Law

A mathematician named De Morgan developed a pair of important laws regarding group complementation in Boolean algebra.

Multiplicative form

$\overline{A.B}=\overline{A}+\overline{B}$

A	*B*	*AB*	$\overline{A.B}$	$\overline{A}$	$\overline{B}$	$\overline{A}+\overline{B}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

Additive form

$\overline{A+B}=\overline{A}.\overline{B}$

A	*B*	*A+B*	$\overline{A+B}$	$\overline{A}$	$\overline{B}$	$\overline{A}.\overline{B}$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Above highlighted columns are identical.

5. Double Complement Law

$A=\overline{\overline{A}}$

A	$\overline{A}$	$\overline{\overline{A}}$
0	1	0
0	1	0

6. Commutative Law

Multiplicative form

$AB = BA$

A	B	AB	BA
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

Additive form

$A + B = B + A$

A	B	A+B	B+A
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

Above highlighted columns are identical.

7. Associative Law

Multiplicative form

A(BC)=(AB)C

A	B	C	BC	AB	A(BC)	(AB)C
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	1	0	0	0
1	0	0	0	0	0	0
1	0	1	0	0	0	0
1	1	0	0	1	0	0
1	1	1	1	1	1	1

Above highlighted columns are identical.

Additive form.

A + (B + C) = (A + B) + C

A	B	C	B+C	A+B	A+(B+C)	(A+B)+C
0	0	0	0	0	0	0
0	0	1	1	0	1	1
0	1	0	1	1	1	1
0	1	1	1	1	1	1
1	0	0	0	1	1	1
1	0	1	1	1	1	1
1	1	0	1	1	1	1
1	1	1	1	1	1	1

Above highlighted columns are identical.

8. Distributive Law

A (B+C) = AB+AC

A	B	C	B+C	AB	AC	A (B+C)	AB+AC
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	1	0	1	1	1
1	1	0	1	1	0	1	1
1	1	1	1	1	1	1	1

Above highlighted columns are identical.

9. Redundancy Law

Form 1

A + AB =A

A	B	AB	A+AB
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

Above highlighted columns are identical.

Form 2

$A + \overline{A}B = A + B$

A	B	A'	A'B	A+A'B	A+B
0	0	1	0	0	0
0	1	1	1	1	1
1	0	0	0	1	1
1	1	0	0	1	1

Simplifying logic expressions using Boolean theorems

E.g. 01 Q = ABC + A'BC + $AB\overline{C}$ = AB (C+$\overline{C}$) + $\overline{A}$BC Distributive Law = AB (1) + $\overline{A}$BC Inverse Law = AB + $\overline{A}$BC Distributive Law = B (A+$\overline{A}$C) Distributive Law = B (A+C) Redundancy Law	E.g. 03 F=C+ $\overline{BC}$ = C+ $\overline{B}$ +$\overline{C}$ - De Morgan's Law = C + $\overline{C}$ + $\overline{B}$ = 1+$\overline{B}$ - Inverse Law = 1 - Identity Law
E.g. 02 F = (A+B) (A+C) = AA + AC + BA + BC - Distributive Law = A+ AC +BA + BC - Idempotent Law = A (1+C+B) + BC - Distributive Law = A (1+B) + BC - Identity Law = A (1) + BC - Identity Law = A + BC - Distributive Law	E.g. 04 F= X(X+Y) = X.X + X.Y - Distributive Law = X + X.Y - Idempotent Law = X (1+Y) - Distributive Law = X (1) - Null/ Identity Law = X - Distributive Law

Standard forms in Boolean expressions

There exist two standard forms of Boolean expressions

-SOP (Sum Of Products)

-POS (Product of Sum)

The Sum-of-Products (SOP) Form

- An SOP expression → when two or more product terms are summed by Boolean summation.
  - Examples:

    $AB + ABC$

    $ABC + CDE + \overline{B}C\overline{D}$

    $\overline{A}B + \overline{A}B\overline{C} + AC$

  - Also:

    $A + \overline{A}\overline{B}C + BC\overline{D}$

In an SOP form, a single overbar cannot extend over more than one variable; however, more than one variable <u>in a term</u> can have an overbar:

example: $\overline{A}\overline{B}\overline{C}$ is correct

But $\overline{ABC}$ is not correct

- A standard SOP expression is one in which *all* the variables in the domain appear <u>in each product term</u> in the expression.
  - Example:

    $A\overline{B}CD + \overline{A}\overline{B}C\overline{D} + AB\overline{C}\overline{D}$

  - Standard SOP expressions are important in:
    - Constructing truth tables
    - The Karnaugh map simplification method

The Product-of-Sum (POS) Form

- When two or more sum terms are multiplied, the result expression is a product-of-sums (POS):
  - Examples:

    $(\overline{A} + B)(A + \overline{B} + C)$

    $(\overline{A} + \overline{B} + \overline{C})(C + \overline{D} + E)(\overline{B} + C + D)$

    $(A + B)(A + \overline{B} + C)(\overline{A} + C)$

  - Also: → $\overline{A}(\overline{A} + \overline{B} + C)(B + C + \overline{D})$

In a POS form, a single overbar cannot extend over more than one variable; however, more than one variable <u>in a term</u> can have an overbar:

example: $\overline{A}+\overline{B}+\overline{C}$ is correct.

However, $\overline{A+B+C}$ is not correct

Minterms and Maxterms

Minterm

Each row of a truth table can be associated with a *minterm,* which is a product (AND) of all variables in the function, in direct or complemented form. A minterm has the property that it is equal to 1 on exactly one row of the truth table.

Maxterm

Each row of a truth table is also associated with a *maxterm,* which is a sum (OR) of all the variables in the function, in direct or complemented form. A maxterm has the property that it is equal to 0 on exactly one row of the truth table.

A	B	C	Z
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$A+B+C$

$\overline{A}\overline{B}C$

$A+\overline{B}+C$

$A+\overline{B}+\overline{C}$

$\overline{A}+B+C$

$A\overline{B}C$

$AB\overline{C}$

ABC

Maxterm

Minterm

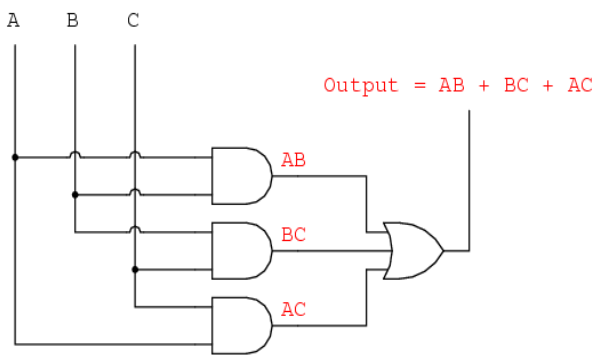
Boolean equation based on SOP method

$$Z=\overline{A}\overline{B}C+A\overline{B}C+AB\overline{C}+ABC$$

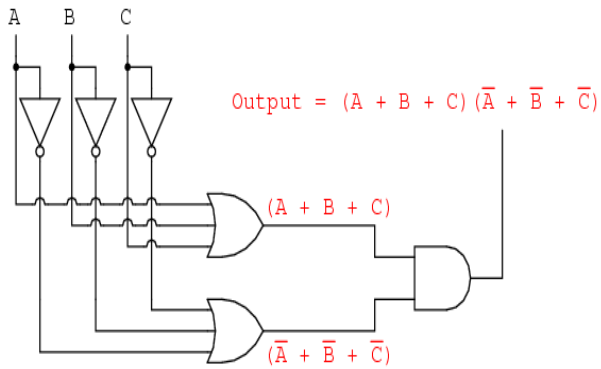
Boolean equation based on POS method

$$Z=(A+B+C)(A+\overline{B}+C)$$
$$(A+\overline{B}+\overline{C})(\overline{A}+B+C)$$

Logic circuit based on SOP



Logic circuit based on POS



Transforms SOP into POS and vice versa

SOP TO POS	
$F = A.\overline{B} + B.\overline{C} + \overline{A}.C$	*Obtain the compliment of whole equation*
$= \overline{A.\overline{B} + B.\overline{C} + \overline{A}C}$	*by adding an over bar to the whole*
$= \overline{A.\overline{B}} + \overline{B.\overline{C}} + \overline{\overline{A}C}$	De Morgan's Law
$= (\overline{A} + \overline{\overline{B}}).(\overline{B} + \overline{\overline{C}}) + (\overline{\overline{A}} + \overline{C})$	De Morgan's Law
$= \underline{(\overline{A} + B).(\overline{B} + C).(A + \overline{C})}$	Double Inverse
POS TO SOP	
$F = (\overline{A} + B).(\overline{B} + C).(A + \overline{C})$	*Obtain the compliment of whole equation*
$= \overline{(\overline{A} + B).(\overline{B} + C).(A + \overline{C})}$	*by adding an over bar to the whole*
$= (\overline{\overline{A} + B}) + (\overline{\overline{B} + C}) + (\overline{A + \overline{C}})$	De Morgan's Law
$= (\overline{\overline{A}}.\overline{B}) + (\overline{\overline{B}}.\overline{C}) + (\overline{A}.\overline{\overline{C}})$	De Morgan's Law
$= \underline{A.\overline{B} + B.\overline{C} + \overline{A}.C}$	Double Inverse

Karnaugh Maps

Karnaugh maps provide an alternative way of simplifying logic circuits.

Instead of using Boolean algebra simplification techniques, you can transfer logic values from a Boolean statement or a truth table into a Karnaugh map.

The arrangement of 0's and 1's within the map helps you to visualize the logic relationships between the variables and leads directly to a simplified Boolean statement.

Cell = 2^n where n is a number of variables

Two input K-Map
Cell = 2^2 =4

B \ A	0	1
0		
1		

Three input K-Map
Cell = 2^3 =8

C \ AB	00	01	11
0			
1			

Four input K-Map
Cell = 2^4 =16

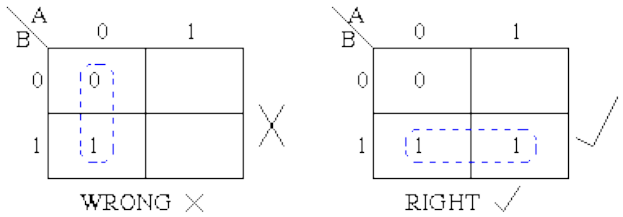
CD \ AB	00	01	11	10
00				
11				
11				
10				

Rules of K- map simplification

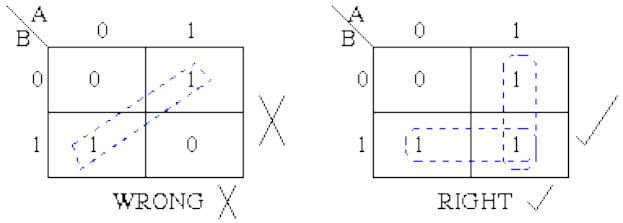
1. No zeros allowed. 2. No diagonals. 3. Only power of 2 number of cells in each group. 4. Groups should be as large as possible.	5. Every “one” must be in at least one group. 6. Overlapping allowed. 7. Wrap around allowed. 8. Fewest number of groups possible.

The Karnaugh map uses the following rules for the simplification of expressions by *grouping* together adjacent cells containing *ones*

- Groups may not include any cell containing a zero

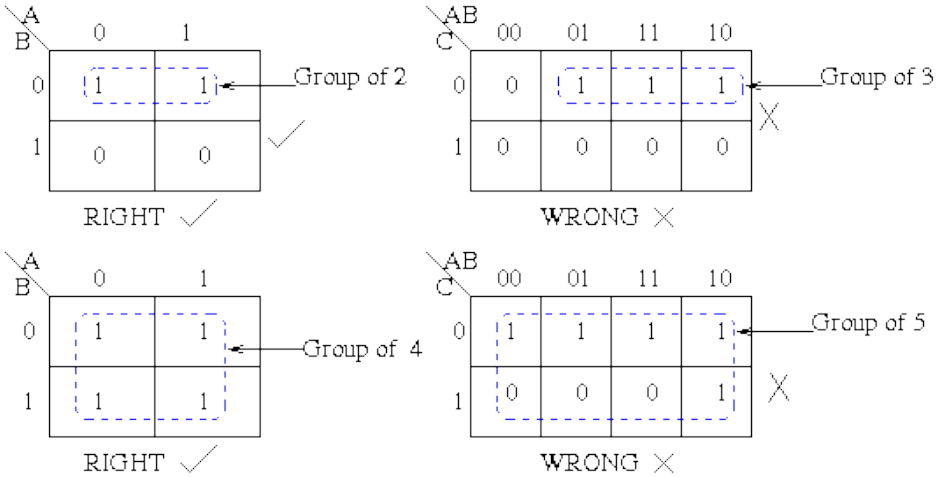


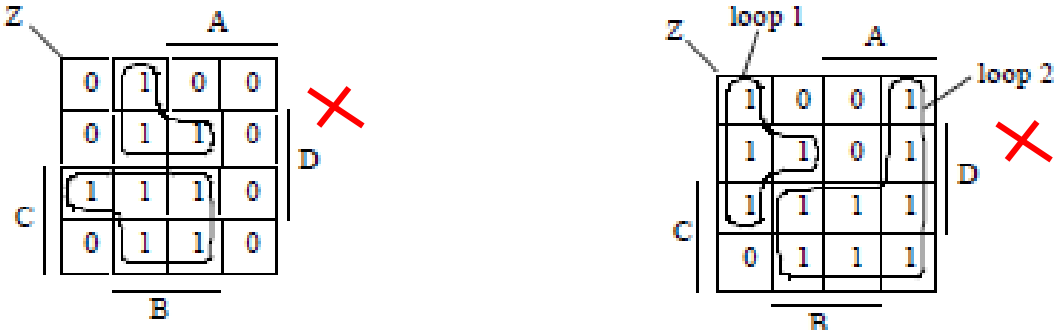
- Groups may be horizontal or vertical, but not diagonal.



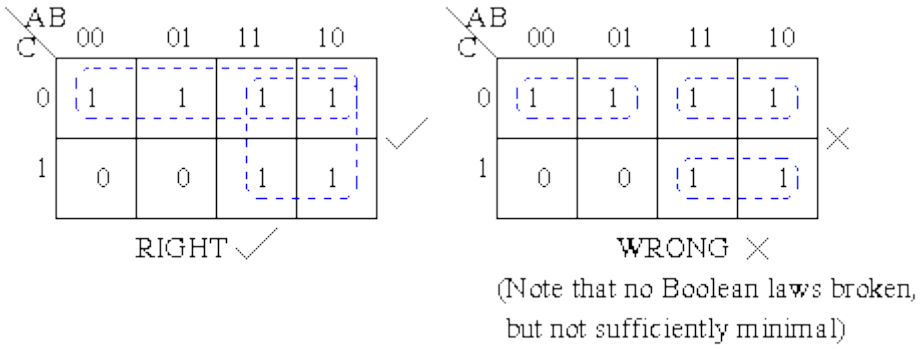
- Groups must contain 1, 2, 4, 8, or in general 2^n cells.

- That is if n = 1, a group will contain two 1's since $2^1 = 2$.
  If n = 2, a group will contain four 1's since $2^2 = 4$.

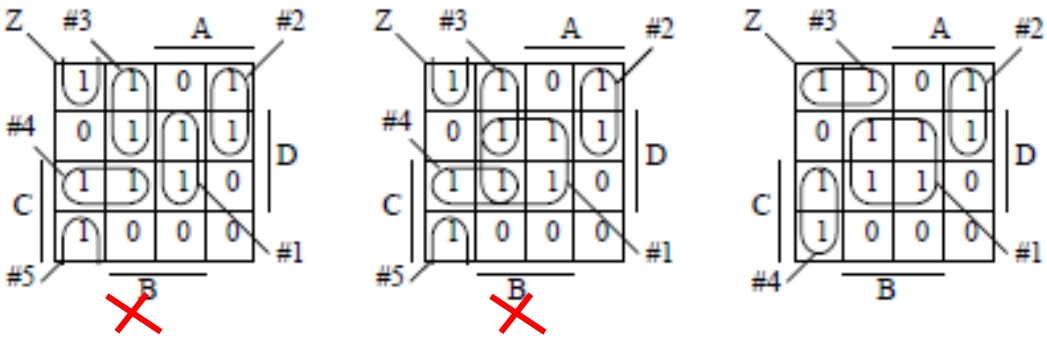




- **Each group should be as large as possible.**



-



- Each cell containing a *one* must be in at least one group.

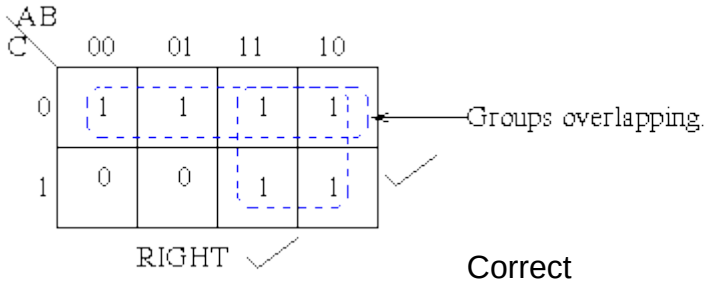
CD \ AB	00	01	11	10
00	0	0	0	1
01	1	1	0	0
11	0	0	1	0
10	0	0	1	0

Group 3

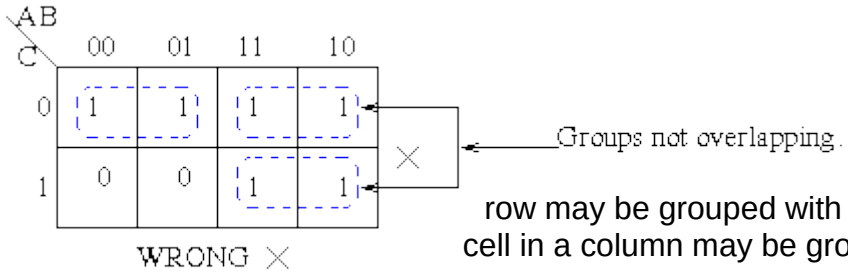
Group 1

Group 2

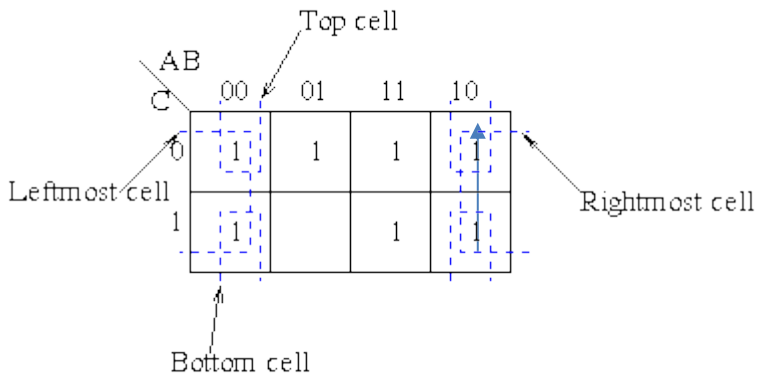
Groups may overlap.



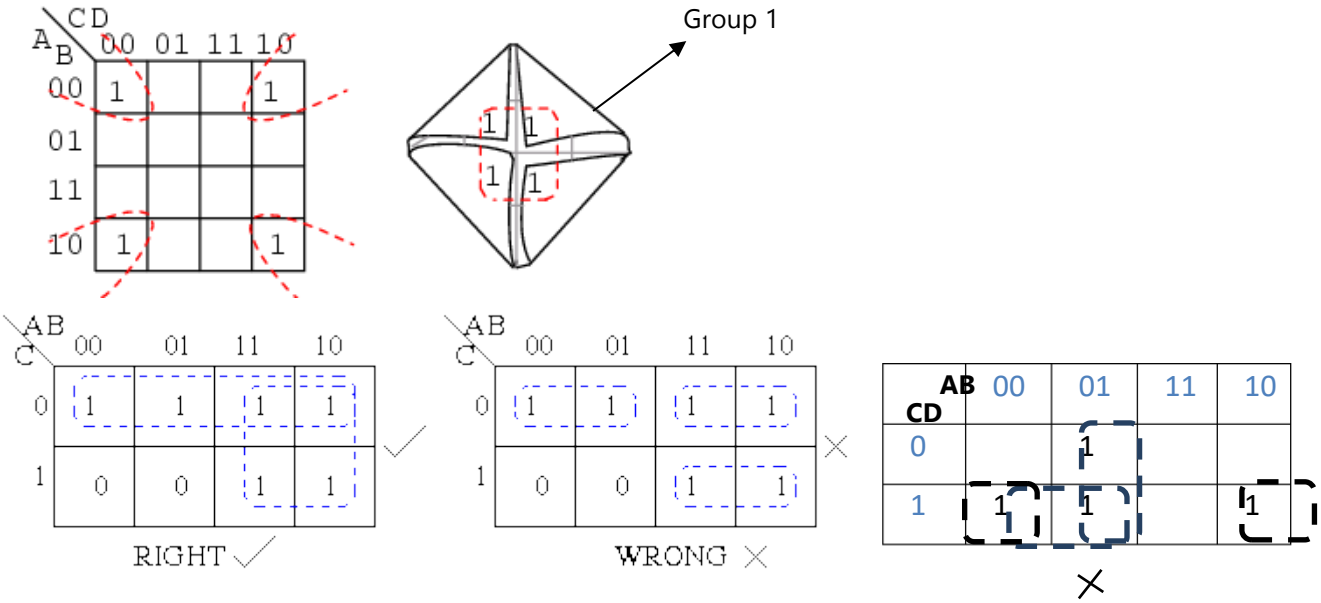
Correct



Groups may wrap around the table. The leftmost cell in a row may be grouped with the rightmost cell and the top cell in a column may be grouped with the bottom cell.



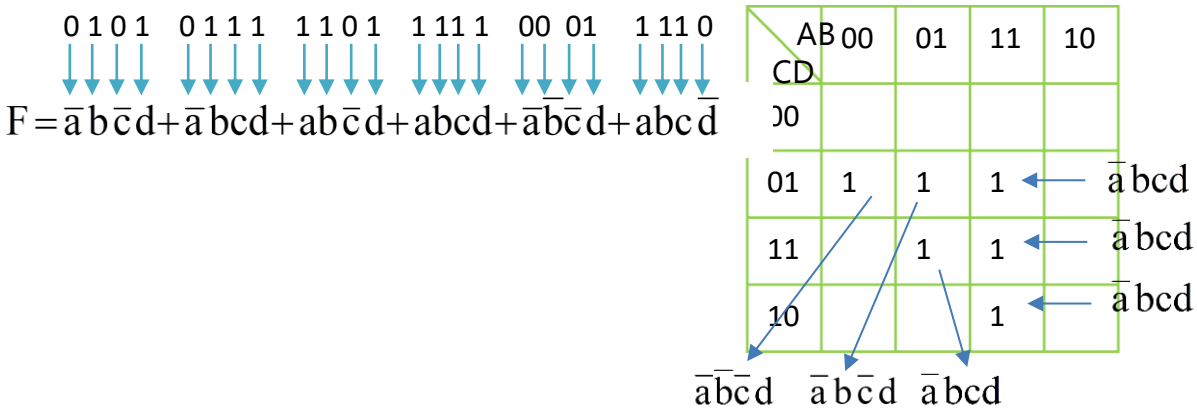
There should be as few groups as possible, as long as this does not contradict any of the previous rules.



How to simplify logic expression using K-maps

How to map four variable logic expression using K-map

In a Boolean expression any variable with a bar, is considered as a 0 and others are considered as 1



K-Map simplification step 2 → Grouping

CD \ AB	00	01	11	10
00	0	0	0	0
01	1	1	1	0
11	0	1	1	0
10	0	0	1	0

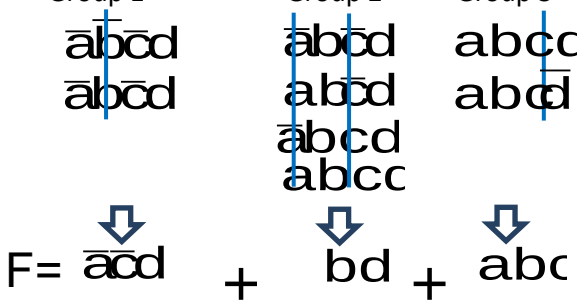
Group 2

Group 3

Group 1

Step 3

Group 1 Group 2 Group 3



$F = \bar{a}\bar{c}d + bd + abc$

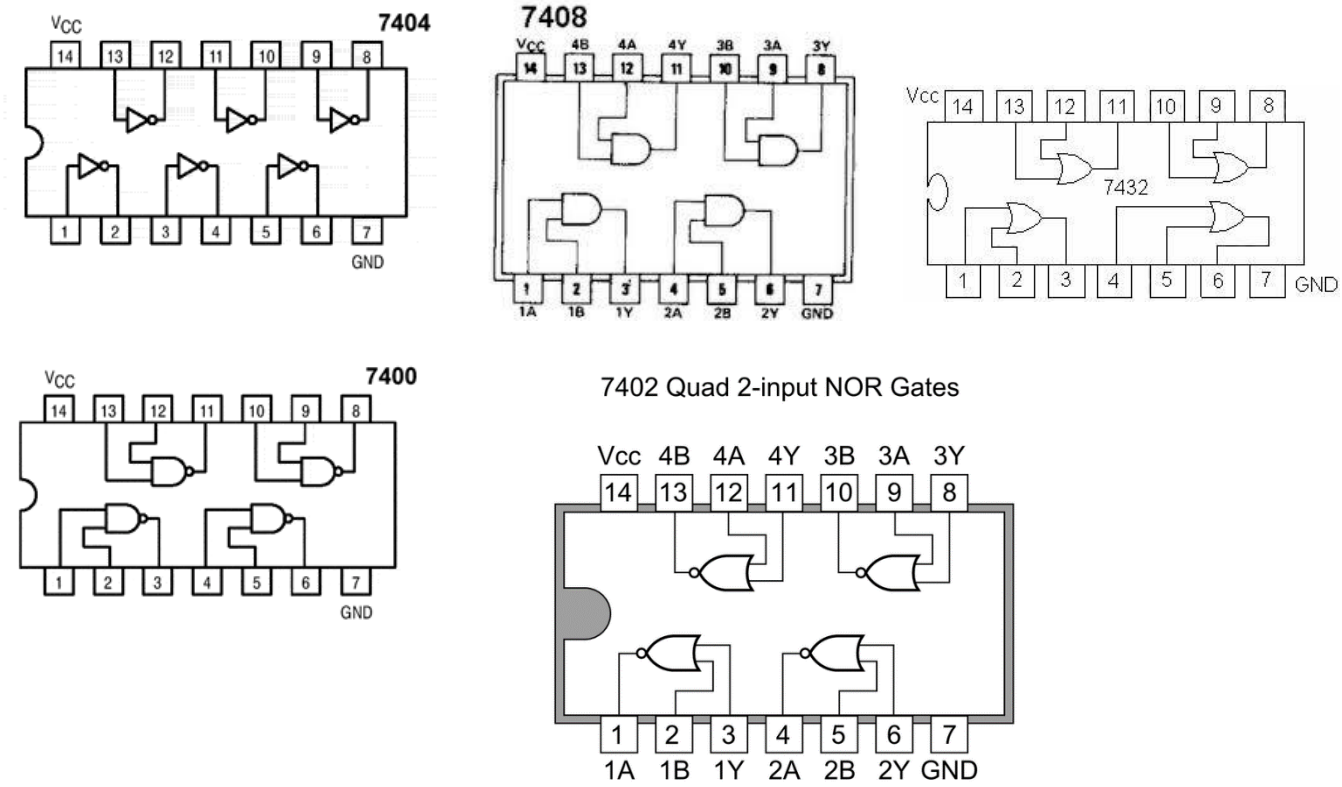
Competency Level 4.3: Designs simple digital circuits using logic gates

Learning outcomes:

- Identifies situation to apply logic circuits in day to day life.
- Design logic expressions and truth table for identified applications
- Designs digital circuits.

Content:

- Truth tables and logic expressions for their designs (up to three inputs)
- Digital circuit design.



Above figure Illustrates the pin connections on three commercially available integrated circuits that implement logic gates. Pins 7 and 14 supply power (zero volts and five volts) to run the circuit.

Design simple logic circuits

E.g. 01 Draw a logic circuit diagram that identifies four-bit prime number.

i. Draw the truth table

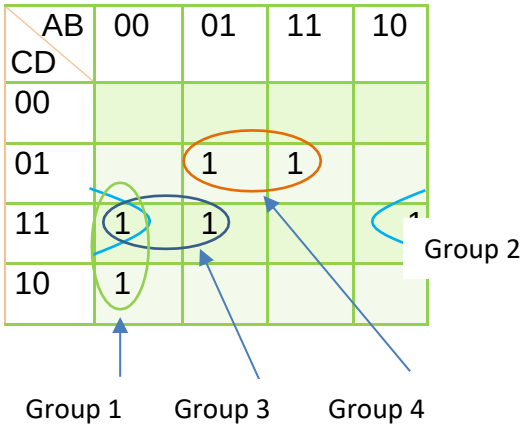
A= First bit, B- Second bit, C- Third bit, D- Fourth bit, F- Prime Number (Output)

	A	B	C	D	F	Minterm
0	0	0	0	0	0	
1	0	0	0	1	0	
2	0	0	1	0	1	A’B’CD’
3	0	0	1	1	1	A’B’CD
4	0	1	0	0	0	
5	0	1	0	1	1	A’BC’D
6	0	1	1	0	0	
7	0	1	1	1	1	A’BCD
8	1	0	0	0	0	
9	1	0	0	1	0	
10	1	0	1	0	0	
11	1	0	1	1	1	AB’CD
12	1	1	0	0	0	
13	1	1	0	1	1	ABC’D
14	1	1	1	0	0	
15	1	1	1	1	0	

ii. Obtain the SOP

F= A’B’CD’ + A’B’CD + A’BC’D + A’BCD + AB’CD + ABC’D

Simplify using K-map



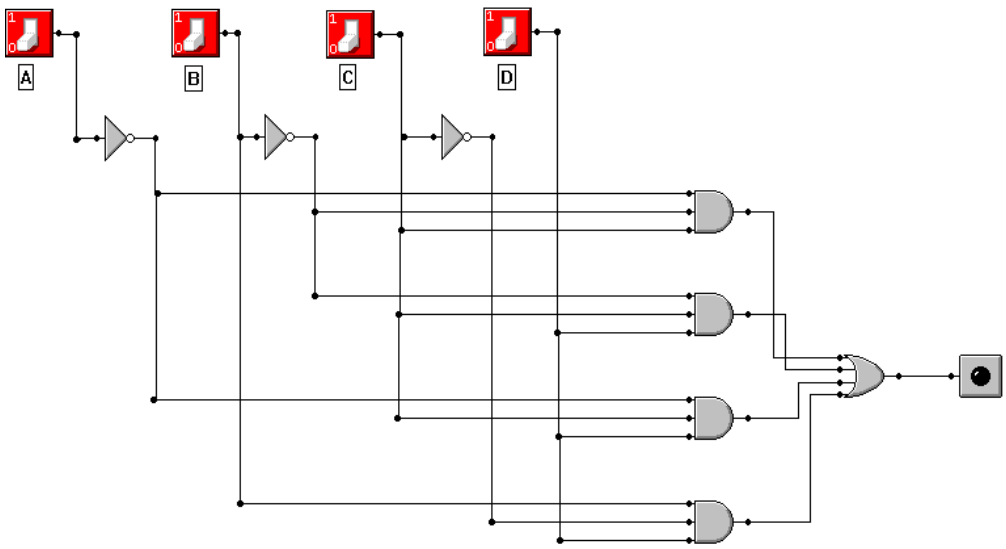
A’B’CD
A’B’CD’

A B’CD
A’B’CD

A’B’CD
A’B CD

A’BC’D
A BC’D

iii. Draw Logic circuit for the simplified version

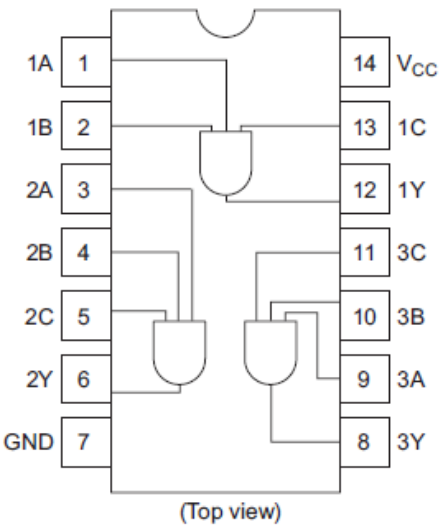


In-order to implement this circuit, following circuitry equipment is required.

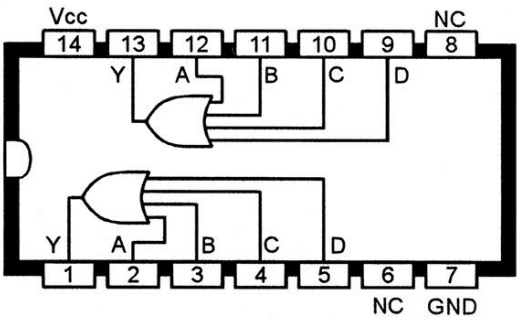
• Project board 1 • ON/OFF Switch 5v DC- 6 • Resistors 1K-10 • Project board jumper wire set 1 • LED Large size-4	• DC 5V Power pack-1 • Logic IC HD74HC11-2 (TRIPPLE THREE INPUT AND GATE) • Logic IC CD4069UBC (HEX INVERTER) • Logic IC TC4072BP (DUAL FOUR INPUT OR GATE)

Pin arrangement

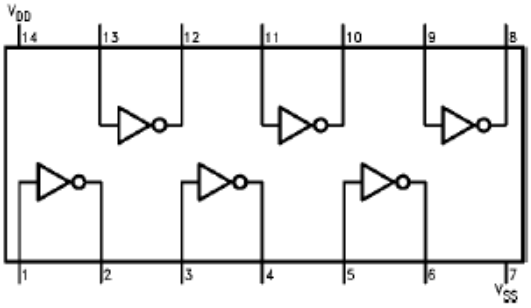
HD74HC11



TC4072BP



CD4069UB



E.g. 02

A lamp outside a front door comes on automatically when it is dark **and** someone stands on the doormat outside the front door.

- A pressure sensor (A) under the mat changes from OFF (0) to ON (1) when someone stands on the doormat.
- The light sensor (B) is ON (1) when it is light and OFF (0) when it is dark.
- There is an on/off switch (C) for a user to manually switch off the system. Consider switching off process as Boolean (1) and switching on as Boolean (0).

- Based on the following conditions the lamp is in off state.

    - Any time user manually switches off the system: lamp (0)
    - When it is not dark: lamp (0).

A (pressure sensor)	B (Light sensor)	C (Switch)	X (Lamp)	SOP
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	0	
1	0	0	1	AB'C'
1	0	1	0	
1	1	0	0	
1	1	1	0	

Prepare a truth table that represents the above scenario.

i. Write the standard Sum of Product (SOP) that represents the above requirement. X=AB'C'

Example 3:

1. A bank vault has 3 locks with a key for each lock. Key A is owned by the bank manager. Key B is owned by the senior bank teller. Key C is owned by the trainee bank teller. In order to open the vault door at least two people must insert their keys into the assigned locks at the same time. The trainee bank teller) can only open the vault when the bank manager is present in the opening.

    i) Determine the truth table for such a digital locking system
    X= Bank Manager
    Y= Senior Bank teller
    Z= Trainee bank teller

X	Y	Z	Vault (F)	Minterm
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	0	
1	0	0	0	
1	0	1	1	XY'Z
1	1	0	1	XYZ'
1	1	1	1	XYZ

ii) Design, using Boolean algebra techniques, a minimum AND-OR gate network to realize this locking system.

F= XY'Z+XYZ'+XYZ
= XY'Z + XY (Z' + Z) Distributive Law
= XY'Z + XY (1) Inverse Law
= XY'Z + XY Distributive Law
= X (Y'Z + Y) Distributive Law
= <u>X (Z + Y) Redundancy Law</u>

E.g. 04

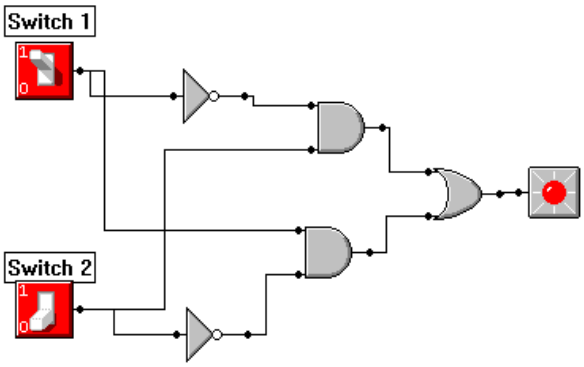
A house has two lights to illuminate the stairs leading from the hall to the upstairs landing. The lights can be switched OFF and ON by either one of two switches, one in the hall and one in the landing. The lights are to be OFF when both switches are either ON or OFF together, and the lights are to be ON when one switch is ON and the other is OFF.

i) Obtain the truth table of the system

A (Switch 1)	B (Switch 2)	F (Bulb)	Minterm
0	0		
0	1	1	A'B
1	0	1	AB'
1	1		

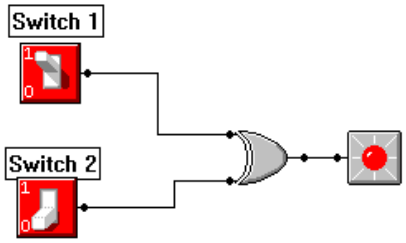
ii) Obtain the sum of products (SOP) Boolean expression for the system.
A'B + AB'

iii) Implement the system using AND, NOT and OR logic gates.



iv) Implement the system using exactly ONE logic gate.

XOR Gate



Competency 4.4: Explores how combinational logic circuits are used in CPU and sequential circuits in physical

Number of periods: 06

Learning Outcomes:

- Identifies the major building blocks of CPU
- Creates truth table and logical expression for half adder circuit
- Creates truth table and logical expression for full adder circuit
- Briefly describes the usage of flip-flops.

Contents:

- Building blocks of CPU
    - Half Adder
    - Full Adder
- Storing bits in digital circuits
    - Feedback loop
    - Flip=flops

Concepts and terms to be highlighted:

- CPU
- Half Adder
- Full Adder
- Storing bits in digital circuits, feedback loop, flip flops

Guidance for lesson plans:

- provide some example of the usage of number systems in computers (colour code etc.)
- Then discuss the Half adder, full adder, storing bits in digital circuits, feedback loop and flip flops to provide the concept about the usage of binary number system in computers

Guidance for assessments and evaluations:

- Divide the students into groups and ask the each group to find the usage of number systems in computer and present the findings.

Quality inputs:

- Computers with some graphic software, Internet facility

Reading Materials

Half Adder

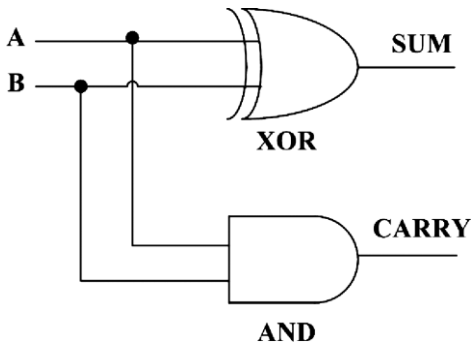
- With the help of half adder, we can design circuits that are capable of performing simple addition with the help of logic gates.
- Let us first take a look at the addition of single bits.
    - 0+0 = 0
    - 0+1 = 1
    - 1+0 = 1
    - 1+1 = 10

These are the least possible single-bit combinations. However, the result for 1+1 is 10. Therefore, the sum result must be re-written as a 2-bit output. Thus, the above equations can be written as

0+0 = 00
0+1 = 01
1+0 = 01
1+1 = 10

Here the output '1'of '10' becomes the carry-out. The result is shown in a truth-table below. 'SUM' is the normal output and 'CARRY' is the carry-out.

INPUTS		**OUTPUTS**	
A	**B**	**SUM**	**CARRY**
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Full Adder

The main difference between a half-adder and a full-adder is that the **full-adder has three inputs and two outputs**. The first two inputs are A and B and the third input is an input carry designated as Carry In. When a full adder logic is designed we will be able to string eight of them together to create a 8- bit adder and cascade the carry bit from one adder to the next.

The carry is designated as Carry Out and the sum is designated as Sum. The truth-table is given below.

Example: Application of half adder and full adder in number addition is shown below. There are two bits additions and three bits addition.

Number 1		1	0	1	1
Number 2		0	1	1	0 +
Carry In	1	1	1	0	
Sum	1	0	0	0	1
Carry Out		1	1	1	0

In here when first two bits are added only half adder is required because unavailability of carry in bit. In-order to add rest of the bits, full adder is required because availability of carry in bit. Full adder is a combination of two half adders.

How to represent a full adder from truth table and a logic circuit?

The carry can be obtained by A.B + B.C + A.C. The sum can be obtained by A XOR B XOR C. The truth table is as follows.

1	2	3	4	5	6	7	8	9	10	11
A	B	C	Sum	Carry	A.B	B.C	A.C	A.B+B.C+A.C	A XOR B	A XOR B XOR C
				Carry In				Sum		Carry Out
0	0	0	0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0	0	0	1
0	1	0	1	0	0	0	0	0	1	1
0	1	1	0	1	0	1	0	1	1	0
1	0	0	1	0	0	0	0	0	1	1
1	0	1	0	1	0	0	1	1	1	0
1	1	0	0	1	1	0	0	1	0	0
1	1	1	1	1	1	1	1	1	0	1

The above truth table shows output values for all alternative input values. The columns of number 5 and 9 are identical. Then the sum can be taken from the Boolean expression of A.B+B.C+A.C. Similarly, the columns of number 5 and 11 are identical. Therefore, carry out can be taken from A XOR B XOR C. Then the logic circuit for the full adder is as follows.